

Zincirler (Chains) Teknik Çalışma Raporu

YÖNETİCİ ÖZETİ: Bu çalışma raporu, Harrison Chase (LangChain Kurucusu) ve Andrew Ng tarafından sunulan **'LangChain for LLM Application Development'** eğitiminin en kritik bölümü olan **'Chains' (Zincirler)** konusuna odaklanmaktadır. Raporda; en basit LLM zincirlerinden, ardışık mantıksal akışlara (Sequential) ve gelişmiş dinamik yönlendirme mekanizmalarına (Router) kadar tüm zincir yapıları; teorik altyapısı, günlük hayattan örnek analogileri ve doğrudan uygulanabilir Python kod örnekleriyle detaylı olarak incelenmiştir.

Giriş ve Zincir (Chain) Kavramı

Büyük Dil Modelleri (LLM), tek başlarına çalıştırıldıklarında oldukça güçlüdür. Ancak karmaşık ve gerçek dünya senaryolarında, tek bir prompt (istem) genellikle tüm bir iş sürecini çözmek için yetersiz kalır. İşte bu noktada **LangChain** devreye girerek bu sistemlerin en önemli yapı taşı olan **'Zincirleri' (Chains)** sunar.

Bir zincir, temel olarak **bir dil modelini (LLM) bir prompt şablonu ile birleştirir**. Bu temel yapı taşları, metin veya diğer veri türleri üzerinde ardışık işlemler gerçekleştirmek üzere birbirine eklenebilir. LangChain'in gücü, bu zincirleri tek bir girdi üzerinde çalıştırmakla sınırlı kalmayıp, aynı anda geniş veri setleri (örneğin pandas DataFrame yapıları) üzerinde satır satır ve otomatik olarak koşturabilmesinden gelmektedir.

▮ **GÜNLÜK HAYATTAN ANALOJİ (Fabrika Montaj Hattı):** Bir zinciri, bir fabrikanın montaj hattına benzetebiliriz. Hattaki her iş istasyonu (zincir adımı) kendine gelen yarı mamul ürünü (girdi) alır, üzerine kendi işlemini ekler (LLM çağırısı / Prompt formatlama) ve bir sonraki istasyona aktarır. Son istasyondan çıkan ürün ise nihai çıktımızdır.

Bölüm 1: LLM Zinciri (LLM Chain) - En Temel Yapı Taşı

LLM Chain (LLM Zinciri), LangChain'in en basit ama gelecekte oluşturulacak tüm gelişmiş zincirlerin temelinde yer alan en güçlü zincir türüdür. Bu yapı, bir prompt şablonu (Prompt Template) ile bir dil modelini (LLM) birleştirir. Girdiyi alır, prompt şablonunu arka planda formatlar, LLM'e gönderir ve sonucu döndürür.

Gerçek Dünya Senaryosu: Şirket İsmi Bulucu

Bir girişimcisiniz ve yeni bir ürün üretiyorsunuz. Bu ürüne en uygun şirket ismini bulmak istiyorsunuz. Örneğin, ürünümüz bir **'queen-size-sheet-set' (kraliçe boy nevresim takımı)** olsun. Zincirimiz, ürün tanımını alıp bize anlamlı ve akılda kalıcı bir şirket ismi olan **'Royal Bettings'** önerisini otomatik olarak dönecektir.

```
# Gerekli kütüphanelerin içe aktarılması
from langchain.chat_models import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain.chains import LLMChain

# Dil modelinin yüksek sıcaklık (temperature) ile yaratıcı olması için ilklendirilmesi
llm = ChatOpenAI(temperature=0.9)

# Prompt şablonunun oluşturulması (Girdi değişkeni: product)
prompt = ChatPromptTemplate.from_template(
    "Bu ürünü üreten bir şirket için en iyi isim nedir? Ürün: {product}"
)

# LLM ve Prompt'un LLMChain içinde birleştirilmesi
chain = LLMChain(llm=llm, prompt=prompt)

# Zincirin bir ürün açıklamasıyla çalıştırılması
product_desc = "queen-size-sheet-set"
response = chain.run(product_desc)
print(response) # Çıktı: Royal Bettings (veya benzer yaratıcı bir isim)
```

Bölüm 2: Basit Ardışık Zincir (Simple Sequential Chain)

Gerçek hayattaki işler genellikle tek adımlı değildir. Bir adımın çıktısı, diğer adımın girdisi olur. **Simple Sequential Chain (Basit Ardışık Zincir)**, adımların sırayla birbirine bağlandığı ve **tek bir girdi ile tek bir çıktının** olduğu senaryolar için mükemmeldir.

Gerçek Dünya Senaryosu: Şirket Kurulumu ve Açıklama Metni Yazımı

Yukarıdaki şirket ismi bulucu örneğini bir adım daha ileri taşıyalım. Önce ürün açıklamasından bir şirket ismi bulacağız (1. Zincir), ardından bu bulduğumuz şirket ismi için 20 kelimelik akıcı bir kurumsal tanıtım yazısı hazırlatacağız (2. Zincir). Bu iki zincir ardı ardına çalışarak tek bir süreç haline gelir.

```
from langchain.chains import SimpleSequentialChain

# Zincir 1: Üründen Şirket İsmi Üretme
prompt_1 = ChatPromptTemplate.from_template(
    "Bu ürünü üreten şirket için en iyi isim nedir? Ürün: {product}"
)
chain_1 = LLMChain(llm=llm, prompt=prompt_1)

# Zincir 2: Şirket İsminden 20 Kelimelik Tanıtım Yazısı Üretme
prompt_2 = ChatPromptTemplate.from_template(
    "Bu şirket için 20 kelimelik bir açıklama yazın: {company_name}"
)
chain_2 = LLMChain(llm=llm, prompt=prompt_2)

# Zincirlerin sırayla birbirine bağlanması
overall_simple_chain = SimpleSequentialChain(
    chains=[chain_1, chain_2],
    verbose=True
)

# Çalıştırma (Girdi doğrudan Zincir 1'e gider, çıktısı otomatik Zincir 2'ye aktarılır)
overall_simple_chain.run("queen-size-sheet-set")
```

Bölüm 3: Gelişmiş Ardışık Zincir (Sequential Chain)

Eğer iş akışımızda **birden fazla girdi** veya **birden fazla çıktı** bulunuyorsa, basit ardışık zincir yetersiz kalır. Bu durumlarda genel amaçlı **Sequential Chain (Gelişmiş Ardışık Zincir)** kullanılır. Bu yapıda, ara adımların girdileri ve çıktı anahtarları (input_keys / output_keys) çok hassas bir şekilde eşleştirilmelidir.

Gerçek Dünya Senaryosu: Çok Dilli Müşteri Yorum Analiz ve Yanıt Sistemi

Bir e-ticaret siteniz olduğunu ve farklı ülkelerden yorumlar aldığınızı hayal edin. Akışımız şu şekildedir:

- Zincir 1 (Çeviri):** Müşterinin orijinal yorumunu (örn. Fransızca) alır ve İngilizceye çevirir.
- Zincir 2 (Özet):** İngilizceye çevrilmiş yorumu alarak tek cümlelik bir özet çıkarır.
- Zincir 3 (Dil Tespiti):** Orijinal yorumun hangi dilde yazıldığını tespit eder.
- Zincir 4 (Yanıt):** Özet ve tespit edilen dili alarak müşteriye kendi dilinde kibar bir takip mesajı hazırlar.

Değişken Akış Şeması ve Anahtarlar

Zincir No / İşlev	Girdi Değişkeni (Input Key)	Çıktı Değişkeni (Output Key)	Kaynak Girdi
1. Çeviri Zinciri	review (Orijinal Yorum)	English_review	Kullanıcı Girdisi
2. Özetleme Zinciri	English_review	summary	1. Zincir Çıktısı
3. Dil Tespiti Zinciri	review (Orijinal Yorum)	language	Kullanıcı Girdisi
4. Takip Yanıtı Zinciri	summary, language	followup_message	2. ve 3. Zincir Çıktısı

```
from langchain.chains import SequentialChain

# Zincir 1: İngilizceye Çeviri
prompt_1 = ChatPromptTemplate.from_template("Translate the following review to English:\n{review}")
chain_1 = LLMChain(llm=llm, prompt=prompt_1, output_key="English_review")

# Zincir 2: Özet Çıkarma
prompt_2 = ChatPromptTemplate.from_template("Summarize the following review in 1 sentence:\n{English_review}")
chain_2 = LLMChain(llm=llm, prompt=prompt_2, output_key="summary")

# Zincir 3: Dil Algılama
prompt_3 = ChatPromptTemplate.from_template("What language is the following review written in?\n{review}")
chain_3 = LLMChain(llm=llm, prompt=prompt_3, output_key="language")

# Zincir 4: Orijinal Dilde Takip Yanıtı Yazma
prompt_4 = ChatPromptTemplate.from_template(
    "Write a polite response to the following summary in the specified language:\nSummary: {summary}\nLanguage: {language}"
)
chain_4 = LLMChain(llm=llm, prompt=prompt_4, output_key="followup_message")

# Tüm zincirlerin SequentialChain altında birleştirilmesi
overall_chain = SequentialChain(
    chains=[chain_1, chain_2, chain_3, chain_4],
    input_variables=["review"],
    output_variables=["English_review", "summary", "followup_message"],
    verbose=True
)
```

Bölüm 4: Yönlendirici Zincir (Router Chain) - Koşullu Akış

Eğer sabit bir akış yerine, kullanıcının girdisine göre **dinamik olarak farklı yollara/zincirlere sapmak** istiyorsak **Router Chain (Yönlendirici Zincir)** kullanırız. Bu zincir, gelen sorunun konusunu analiz eder ve en uzman alt zincire yönlendirir. Eğer soru tanımlı hiçbir konuya uymuyorsa, otomatik olarak genel amaçlı bir **Varsayılan Zincire (Default Chain)** aktarılır.

Yönlendirici Zincir Bileşenleri

- **Multi-Prompt Chain:** Birden fazla uzman prompt şablonu arasında dinamik geçiş sağlayan ana zincirdir.
- **LLM Router Chain:** Girdiyi analiz edip hangi alt zincirin en uygun olduğunu seçen, karar mekanizması olan LLM'dir.
- **Router Output Parser:** LLM'in verdiği kararı analiz ederek, hedef zincir ismini ve girdiyi içeren bir sözlük yapısına dönüştürür.
- **Varsayılan Zincir (Default Chain):** Biyoloji gibi sisteme tanımlanmamış genel konular geldiğinde devreye giren emniyet supabıdır.

```

from langchain.chains.router import MultiPromptChain
from langchain.chains.router.llm_router import LLMRouterChain, RouterOutputParser
from langchain.chains.router.multi_prompt_prompt import MULTI_PROMPT_ROUTER_TEMPLATE

# 1. Konuya Özel Uzman Şablonların Tanımlanması
prompt_infos = [
    {"name": "physics", "description": "Good for answering physics questions", "prompt_template": "You are a physics expert."},
    {"name": "math", "description": "Good for answering math questions", "prompt_template": "You are a math expert."}
]

# 2. Hedef Zincirlerin (Destination Chains) Oluşturulması
destination_chains = {}
for p_info in prompt_infos:
    name = p_info["name"]
    prompt = ChatPromptTemplate.from_template(p_info["prompt_template"])
    destination_chains[name] = LLMChain(llm=llm, prompt=prompt)

# 3. Varsayılan (Default) Zincirin Tanımlanması
default_prompt = ChatPromptTemplate.from_template("{input}")
default_chain = LLMChain(llm=llm, prompt=default_prompt)

# 4. Yönlendirici (Router) Şablonunun Kurulması
destinations = [f"{{p['name']}}: {{p['description']}}" for p in prompt_infos]
destinations_str = "\n".join(destinations)
router_template = MULTI_PROMPT_ROUTER_TEMPLATE.format(destinations=destinations_str)

router_prompt = ChatPromptTemplate.from_template(
    template=router_template,
    output_parser=RouterOutputParser()
)
router_chain = LLMRouterChain.from_llm(llm, router_prompt)

# 5. Ana MultiPromptChain'in İnşa Edilmesi
chain = MultiPromptChain(
    router_chain=router_chain,
    destination_chains=destination_chains,
    default_chain=default_chain,
    verbose=True
)

# Test: Fizik sorusu fizik zincirine, biyoloji sorusu ise default zincire yönlendirilir.
chain.run("Kara deliklerin olay ufku nedir?") # Fizik zincirine gider
chain.run("Hücre bölünmesi nasıl gerçekleşir?") # Default zincire gider

```

Sonuç ve Değerlendirme

LangChain zincirleri (Chains), büyük dil modellerini salt sohbet robotları olmaktan çıkarıp, belirli mantıksal kurallara göre çalışan, hata payı düşük ve otomatize edilmiş iş istasyonlarına dönüştürür. En basit **LLMChain** ile başlayan serüven, **SequentialChain** ile karmaşık süreçleri birbirine bağlayabilmemize, **RouterChain** ile ise tamamen akıllı ve dinamik kararlar alabilen akışlar tasarlayabilmemize olanak tanır. Bu yapılar modern AI tabanlı uygulama geliştirmenin vazgeçilmez temel yapı taşlarıdır.